



LUCAS HENRIQUE LOPES COSTA

**RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DE
APLICAÇÕES WEB NA EMPRESA ZEEWAY**

LAVRAS – MG

2026

LUCAS HENRIQUE LOPES COSTA

**RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DE APLICAÇÕES WEB NA
EMPRESA ZEEWAY**

Relatório técnico apresentada à Universidade Federal de Lavras, como parte das exigências do programa de Graduação em Sistemas de informação, para a obtenção do título de Bacharel em Sistema de Informação

Prof. Paulo Afonso Parreira Júnior
Orientador

**LAVRAS – MG
2026**

**Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos da Biblioteca
Universitária da UFLA, com dados informados pelo(a) próprio(a) autor(a).**

Costa, Lucas Henrique Lopes

Relatório Técnico de Desenvolvimento de Aplicações Web
na Empresa Zeeway / Lucas Henrique Lopes Costa. – Lavras :
UFLA, 2026.

23p. : il.

Trabalho de Conclusão de Curso (graduação)–Universidade
Federal de Lavras, 2026.

Orientador: Prof. Paulo Afonso Parreira Júnior.

Bibliografia.

1. Desenvolvimento web. 2. React. 3. TypeScript. 4. Scrum.
I. Universidade Federal de Lavras. II. Título.

CDD [Número de Classificação]

LUCAS HENRIQUE LOPES COSTA

**RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DE APLICAÇÕES WEB NA
EMPRESA ZEEWAY**

Relatório técnico apresentada à Universidade Federal de Lavras, como parte das exigências do programa de Graduação em Sistemas de informação, para a obtenção do título de Bacharel em Sistema de Informação

APROVADA em _____ de _____ de _____.

Prof. Paulo Afonso Parreira Júnior
Orientador

**LAVRAS – MG
2026**

Dedico este trabalho a Deus, que guiou os meus passos. Aos meus pais, Lidiana e Antônio, ao meu irmão Marcos Túlio, e a toda a minha família, que sempre me deram apoio para chegar onde estou.

AGRADECIMENTOS

Agradeço primeiramente a Deus, por me dar saúde, resiliência e por guiar os meus passos ao longo de toda esta jornada universitária.

Aos meus pais, Lidiana e Antônio, e ao meu irmão, Marcos Túlio, o meu muito obrigado por serem a minha base. Gratidão pelo apoio incondicional de vocês.

Um agradecimento muito especial aos meus avós, Maria Aparecida e Waldeli; e meus padrinhos, Alessandra e Hélio, por todo o incentivo. Às minhas primas de coração, Fernanda, Júlia e Laura, agradeço por estarem sempre ao meu lado, me acompanhando e se preocupando comigo em cada etapa desse processo. Ao meus tios, Kelson, Ana, Kerley, por também estarem comigo nessa trajetória.

Gostaria de expressar minha profunda gratidão ao Luiz Fernando e ao seu pai, José Ari Ferreira. Vocês foram as pessoas que me motivaram a vir para a UFLA, que me deram a primeira grande oportunidade de trabalho e que acreditaram em mim. Tenho um enorme carinho e uma gratidão imensa por tudo o que construímos e vivemos juntos na Zeeway.

Por fim, agradeço ao professor Paulo Afonso pela orientação, paciência e por compartilhar sua experiência para a construção deste trabalho, e a todos os professores do curso que contribuíram para a minha formação profissional e pessoal.

RESUMO

[Resumo a ser preenchido.]

Palavras-chave: desenvolvimento web; React; TypeScript; Scrum; agronegócio.

ABSTRACT

[Abstract to be filled.]

Keywords: web development; React; TypeScript; Scrum; agribusiness.

LISTA DE FIGURAS

LISTA DE TABELAS

SUMÁRIO

1	INTRODUÇÃO	10
1.1	Contextualização	10
1.2	A Empresa Cliente e o Cenário do Projeto	10
1.3	Objetivos do Estágio e do TCC	11
2	FUNDAMENTAÇÃO TEÓRICA	12
2.1	Gestão de Dados no Agronegócio	12
2.2	Metodologias Ágeis: O <i>Framework Scrum</i>	13
2.3	Tecnologias de Desenvolvimento <i>Front-end</i>	13
2.3.1	React e TypeScript	14
2.3.2	Gerenciamento de Estado e Formulários	14
2.3.3	Design de Interface com Material UI (MUI)	15
2.3.4	Visualização de Dados com ApexCharts	15
3	METODOLOGIA E PROCESSO DE DESENVOLVIMENTO	17
3.1	Aplicação do Scrum no Projeto	17
3.2	Levantamento de Requisitos e Regras de Negócio	17
3.3	Arquitetura do Front-end e Estrutura do Projeto	17
4	RESULTADOS: A PLATAFORMA DE PROTOCOLOS	18
4.1	Módulo de Autenticação e Controle de Acesso	18
4.1.1	Gestão de Usuários	18
4.2	Módulo de Registros Agrícolas	19
4.2.1	Listagem de Protocolos	19
4.2.2	Formulário de Cadastro e Edição de Protocolos	19
4.3	Módulo de Análise e Dashboards	20
4.3.1	Geração Automatizada de Gráficos	21
4.3.2	Dashboard Geral	21
5	CONSIDERAÇÕES FINAIS	22
5.1	Avaliação do Produto Desenvolvido	22
5.2	Impacto do Estágio na Formação Acadêmica e Profissional	22
	REFERÊNCIAS	23

1 INTRODUÇÃO

Neste capítulo introdutório, serão apresentados o contexto do projeto desenvolvido durante o estágio supervisionado, a empresa cliente e o cenário do agronegócio que motivou a construção da plataforma de protocolos. Serão também delineados os objetivos do estágio e deste relatório, que visa documentar as atividades realizadas, as tecnologias adotadas e os resultados alcançados.

1.1 Contextualização

O agronegócio é um dos setores mais importantes e dinâmicos da economia brasileira, com participação relevante no Produto Interno Bruto e forte impacto sobre emprego, renda e exportações (Centro de Estudos Avançados em Economia Aplicada; Confederação da Agricultura e Pecuária do Brasil, 2025). Dentro desse cenário, a pesquisa agrícola focada na análise de solos e sementes desempenha papel central para elevar a produtividade e sustentar decisões técnicas no campo. Com o avanço da chamada Agricultura 4.0, a coleta, o armazenamento e a análise de dados tornaram-se indispensáveis para a tomada de decisões mais rápidas e precisas (Empresa Brasileira de Pesquisa Agropecuária, 2019).

Apesar da crescente digitalização, muitas instituições de pesquisa ainda enfrentam o desafio da fragmentação de dados, dependendo de ferramentas manuais e planilhas eletrônicas para registrar suas descobertas. Esse cenário dificulta a rastreabilidade das informações, a aplicação de regras de negócio complexas e, principalmente, a geração de análises visuais rápidas que poderiam acelerar os resultados das pesquisas (Empresa Brasileira de Pesquisa Agropecuária, 2019). É nesse contexto de modernização e centralização de dados que se insere o projeto desenvolvido durante o estágio.

1.2 A Empresa Cliente e o Cenário do Projeto

As atividades de estágio supervisionado foram realizadas na empresa Zeeway, empresa lavrense de tecnologia fundada no ecossistema de inovação da Universidade Federal de Lavras (UFLA) (Zeeway Soluções Digitais, 2026). A Zeeway atua no desenvolvimento de software e na entrega de soluções tecnológicas personalizadas, destacando-se pelo compromisso com a

qualidade dos projetos e com os resultados de seus clientes (Zeeway Soluções Digitais, 2026). Atualmente, a empresa conta com um quadro de 15 a 30 colaboradores diretos e mantém uma base aproximada de 40 clientes diretos e 500 clientes indiretos. O trabalho principal desenvolvido durante o período de estágio foi a construção de uma plataforma web encomendada pela Terra Gerais, empresa do setor do agronegócio especializada em pesquisa agrícola de solos e sementes.

A necessidade da Terra Gerais consistia em abandonar o uso descentralizado de planilhas e adotar uma plataforma própria para registrar os seus protocolos, que são registros detalhados com as informações de safra, componentes do solo e produtividade de grãos. O acesso ao sistema seria distribuído aos colaboradores, exigindo uma arquitetura robusta para lidar com diferentes perfis de usuário e permissões de acesso.

A participação no projeto concentrou-se no desenvolvimento da interface visual e da experiência do usuário (front-end). A aplicação foi construída utilizando a biblioteca React com TypeScript, empregando componentes da interface Material UI (MUI) e integrando a biblioteca ApexCharts para o diferencial do sistema: a geração automatizada de gráficos de análise agrônoma. Todo o ciclo de desenvolvimento foi pautado pela metodologia ágil Scrum, permitindo organização e entregas incrementais.

1.3 Objetivos do Estágio e do TCC

O objetivo principal do estágio foi aplicar, em um ambiente corporativo real, os conhecimentos adquiridos ao longo da graduação, vivenciando a rotina de desenvolvimento de software em equipe e os desafios da engenharia de requisitos.

Este relatório tem como objetivo apresentar de forma estruturada as atividades desempenhadas na Zeeway, detalhando a arquitetura front-end construída, as tecnologias adotadas e a metodologia de trabalho. Busca-se demonstrar também os desafios técnicos enfrentados na tradução de regras de negócio complexas para a interface da plataforma e o impacto da experiência na formação profissional e acadêmica.

2 FUNDAMENTAÇÃO TEÓRICA

A atividade de desenvolvimento de *software* envolve o uso de linguagens de programação, bibliotecas, *frameworks* e metodologias de trabalho para produzir, testar e entregar sistemas. Neste capítulo são apresentados os conceitos e tecnologias que fundamentaram as atividades realizadas durante o estágio, abrangendo o contexto do agronegócio, a metodologia ágil *Scrum* e as tecnologias de desenvolvimento *front-end* utilizadas na construção da plataforma.

2.1 Gestão de Dados no Agronegócio

O agronegócio brasileiro é responsável por uma parcela significativa do Produto Interno Bruto nacional. De acordo com dados do Centro de Estudos Avançados em Economia Aplicada (Cepea), em parceria com a Confederação da Agricultura e Pecuária do Brasil (CNA), o setor tem apresentado crescimento consistente nas últimas décadas, consolidando o país como um dos maiores produtores e exportadores de alimentos do mundo (Centro de Estudos Avançados em Economia Aplicada; Confederação da Agricultura e Pecuária do Brasil, 2025).

Nesse contexto, a pesquisa agrícola desempenha papel fundamental para a manutenção e o aumento da produtividade. Instituições e empresas especializadas em análise de solos e sementes coletam grandes volumes de dados ao longo das safras, incluindo informações sobre composição do solo, tratamentos aplicados, condições climáticas e resultados de produtividade. Tradicionalmente, esses dados são registrados em planilhas eletrônicas e documentos avulsos, o que dificulta a rastreabilidade, a padronização e a análise integrada das informações (Empresa Brasileira de Pesquisa Agropecuária, 2019).

Com o avanço da chamada Agricultura 4.0, a digitalização dos processos agrícolas tornou-se uma tendência crescente. A Agricultura 4.0 é caracterizada pela adoção de tecnologias digitais, como sensores, *Internet das Coisas* (IoT), computação em nuvem e análise de dados, com o objetivo de otimizar a tomada de decisões no campo (Massruhá; Leite *et al.*, 2020). Nesse cenário, plataformas *web* que centralizam o registro e a análise de dados de pesquisa representam uma evolução em relação ao uso descentralizado de planilhas, permitindo que os colaboradores acessem informações atualizadas, apliquem regras de negócio padronizadas e gerem visualizações gráficas de forma automatizada.

2.2 Metodologias Ágeis: O *Framework Scrum*

O *Scrum* é um *framework* de gerenciamento de projetos e desenvolvimento ágil com formato iterativo e incremental. Cada ciclo no *Scrum* é conhecido como *Sprint*, um processo com duração de tempo fixa e com um conjunto de tarefas que devem ser implementadas nesse período. No final de cada *Sprint*, uma nova versão com novas funcionalidades e melhorias do sistema é entregue (Schwaber; Sutherland, 2020).

No *Scrum*, antes do início de uma *Sprint*, ocorre uma reunião de planejamento chamada *planning*. Nela, o *Product Owner* seleciona as tarefas do *Product Backlog*, que é a lista priorizada com todas as funcionalidades e melhorias planejadas para o produto, e as adiciona ao *Sprint Backlog*, definindo o escopo de trabalho da *Sprint*. O *Product Owner* é o responsável por definir a prioridade das tarefas e planejar o lançamento de novas funcionalidades.

Outra figura-chave no processo do *Scrum* é o *Scrum Master*, pessoa responsável por garantir que os conceitos da metodologia estão sendo aplicados de maneira correta e por remover impedimentos que possam atrapalhar o progresso da equipe. A equipe de desenvolvimento, por sua vez, é composta pelos profissionais que efetivamente executam as tarefas planejadas.

Além da *planning*, o *Scrum* prevê reuniões diárias chamadas *daily meetings*, nas quais cada membro da equipe compartilha o que foi feito no dia anterior, o que será feito no dia atual e se há algum impedimento. No final da *Sprint*, ocorrem a *Sprint Review*, na qual o incremento do produto é apresentado, e a *Sprint Retrospective*, na qual a equipe discute o que funcionou bem e o que pode ser melhorado no processo.

No projeto desenvolvido durante o estágio na Zeeway, o *Scrum* foi adotado como metodologia de desenvolvimento, com *Sprints* de duração fixa, reuniões de planejamento e acompanhamento por meio de *dailies*. A adoção dessa metodologia permitiu entregas incrementais ao cliente e uma organização clara das tarefas ao longo do projeto.

2.3 Tecnologias de Desenvolvimento *Front-end*

Nesta seção são apresentadas as principais tecnologias utilizadas no desenvolvimento da interface da plataforma de protocolos. A escolha dessas tecnologias levou em consideração a necessidade de construir uma aplicação *web* interativa, com formulários complexos, tabelas dinâmicas e visualizações gráficas.

2.3.1 React e TypeScript

O *React* é uma biblioteca *JavaScript* de código aberto, criada pelo *Facebook* (atualmente *Meta*), voltada para a construção de interfaces de usuário. Seu principal conceito é a componentização: a interface é dividida em componentes reutilizáveis, cada um responsável por renderizar uma parte específica da tela. Os componentes podem receber dados por meio de propriedades (*props*) e gerenciar seu próprio estado interno utilizando *hooks* como *useState* e *useEffect* (Meta Platforms, 2024).

Uma das vantagens do *React* é o uso do *Virtual DOM* (*Document Object Model*), uma representação virtual da árvore de elementos da página. Quando o estado de um componente é alterado, o *React* compara a versão atual do *Virtual DOM* com a anterior e atualiza apenas os elementos que de fato mudaram, otimizando a performance de renderização.

O *TypeScript* é um superconjunto do *JavaScript* desenvolvido pela *Microsoft* que adiciona tipagem estática à linguagem. Em projetos *React*, o *TypeScript* permite definir tipos para as *props* dos componentes, para os dados recebidos de APIs e para o estado da aplicação, reduzindo erros em tempo de desenvolvimento e facilitando a manutenção do código (Microsoft, 2024).

No projeto da plataforma de protocolos, a combinação de *React* com *TypeScript* foi utilizada para construir todos os componentes da interface. A ferramenta de *build* adotada foi o *Vite*, que oferece um servidor de desenvolvimento com *Hot Module Replacement* (HMR), permitindo que as alterações no código fossem refletidas instantaneamente no navegador durante o desenvolvimento.

2.3.2 Gerenciamento de Estado e Formulários

Em aplicações *React* de maior porte, o gerenciamento do estado da aplicação torna-se um desafio. Para solucionar esse problema, foi utilizada a biblioteca *Zustand*, uma solução leve de gerenciamento de estado global que permite compartilhar dados entre componentes sem a necessidade de passar *props* por múltiplos níveis da árvore de componentes (Kato, 2024).

Para o gerenciamento dos formulários, que constituíam uma parte central da plataforma dado o grande número de campos nos protocolos, foram utilizadas as bibliotecas *Formik* e *Yup*. O *Formik* é uma biblioteca que simplifica a criação de formulários em *React*, cuidando do estado

dos campos, da validação e do envio dos dados. O *Yup* é uma biblioteca de validação de esquemas que permite definir regras de validação de forma declarativa, como campos obrigatórios, tipos de dados esperados e validações condicionais (Palmer, 2024).

A comunicação com a API do servidor foi implementada por meio da biblioteca *Axios*, um cliente HTTP que permite realizar requisições de forma simplificada. Para o gerenciamento de cache e sincronização dos dados do servidor, foi utilizada a biblioteca *React Query* (*TanStack Query*), que automatiza o processo de busca, cache, atualização e invalidação de dados, evitando requisições desnecessárias e mantendo a interface atualizada (Linsley, 2024).

2.3.3 Design de Interface com Material UI (MUI)

O *Material UI* (MUI) é uma biblioteca de componentes *React* que implementa o *Material Design*, sistema de design criado pelo *Google*. A biblioteca fornece um conjunto amplo de componentes prontos, como botões, campos de texto, tabelas, menus, modais e *date pickers*, todos com aparência e comportamento consistentes (MUI, 2024).

Uma das vantagens do *MUI* é a possibilidade de personalização por meio de temas. No projeto da Terra Gerais, foi criado um tema personalizado que definia as cores, tipografia e espaçamentos de acordo com a identidade visual da empresa cliente. Dessa forma, todos os componentes da biblioteca adotavam automaticamente o padrão visual definido, garantindo consistência em toda a plataforma.

Além dos componentes básicos do *MUI*, o projeto utilizou a biblioteca *MUI X Date Pickers* para os campos de seleção de data, recurso frequentemente utilizado nos formulários de protocolos que exigiam o registro de datas de plantio, colheita e emergência. Também foi utilizada a biblioteca *TanStack React Table* para a construção de tabelas com funcionalidades avançadas de ordenação, paginação e filtragem.

2.3.4 Visualização de Dados com ApexCharts

A geração de gráficos e visualizações de dados constituiu o principal diferencial da plataforma em relação ao uso de planilhas. Para essa funcionalidade, foi utilizada a biblioteca *ApexCharts*, uma biblioteca *JavaScript* de código aberto para a criação de gráficos interativos. A integração com o *React* foi realizada por meio do pacote *react-apexcharts* (Chhipa, 2024).

O *ApexCharts* oferece diversos tipos de gráficos, como barras, linhas, pizza, radar e área, além de funcionalidades como *zoom*, *tooltip* e exportação em formato de imagem. Na plataforma de protocolos, foram utilizados principalmente gráficos de barras verticais e horizontais para comparar os resultados entre diferentes tratamentos e momentos de avaliação.

Para complementar a geração dos gráficos, a biblioteca *mathjs* foi utilizada na realização de cálculos estatísticos sobre os dados dos protocolos, como médias e distribuições percentuais. A exportação dos gráficos em formato de imagem, com a identidade visual da Terra Gerais, foi implementada com as bibliotecas *html2canvas* — que realiza a captura de elementos HTML como imagem — e *JSZip*, que permite o empacotamento de múltiplos arquivos em um único arquivo compactado para *download*.

3 METODOLOGIA E PROCESSO DE DESENVOLVIMENTO

3.1 Aplicação do Scrum no Projeto

3.2 Levantamento de Requisitos e Regras de Negócio

3.3 Arquitetura do Front-end e Estrutura do Projeto

4 RESULTADOS: A PLATAFORMA DE PROTOCOLOS

Neste capítulo são descritas as principais funcionalidades desenvolvidas durante o estágio para a plataforma de protocolos da Terra Gerais. A apresentação está organizada por módulo, detalhando as decisões técnicas tomadas e os desafios enfrentados em cada etapa. A seleção das atividades se deu por meio da sua relevância no escopo do projeto e pela complexidade técnica envolvida.

4.1 Módulo de Autenticação e Controle de Acesso

O primeiro módulo desenvolvido foi o de autenticação, responsável por controlar o acesso dos colaboradores da Terra Gerais à plataforma. A tela de login foi construída seguindo a identidade visual da empresa cliente, com campos para e-mail e senha e a opção de recuperação de senha.

A autenticação foi implementada com o uso da biblioteca *jwt-decode* para decodificação de tokens JWT (*JSON Web Token*) recebidos da API. Após o login bem-sucedido, o token era armazenado e utilizado para validar as requisições subsequentes por meio do *Axios*, configurado com interceptadores que adicionavam o cabeçalho de autorização automaticamente.

Um dos desafios deste módulo foi a implementação do controle de permissões por perfil de usuário. A plataforma previa dois perfis principais, Administrador e Técnico, com diferentes níveis de acesso às funcionalidades. O perfil Administrador possuía acesso completo ao sistema, incluindo a gestão de usuários, enquanto o perfil Técnico tinha acesso restrito apenas ao registro e consulta de protocolos. Essa lógica foi implementada no front-end por meio de verificações condicionais nos componentes React, utilizando o estado global gerenciado pela biblioteca *Zustand*.

4.1.1 Gestão de Usuários

A tela de gestão de usuários permitia ao administrador visualizar, adicionar, editar e excluir os colaboradores cadastrados. A listagem exibia nome, tipo de perfil, e-mail e contato de cada usuário, com suporte a busca textual e filtros por tipo de perfil.

As operações de criação e exclusão de usuários eram realizadas por meio de modais, componentes sobrepostos à tela principal que permitem ao usuário executar uma ação sem perder o contexto da página atual. Foram implementados modais para o formulário de adição de usuário, para a confirmação de exclusão e para o *feedback* de sucesso e erro. Todos os formulários utilizavam a biblioteca *Formik* para gerenciamento de estado dos campos e *Yup* para validação dos dados antes do envio à API.

4.2 Módulo de Registros Agrícolas

O módulo de registros agrícolas constituiu o núcleo da plataforma, sendo responsável pelo cadastro, edição e consulta dos protocolos de pesquisa. Cada protocolo representava um registro completo de uma pesquisa agrícola, contendo informações sobre a safra analisada, os tratamentos aplicados, as características do solo e os resultados de produtividade.

4.2.1 Listagem de Protocolos

A tela de listagem de protocolos foi construída utilizando a biblioteca *TanStack React Table*, que oferece recursos avançados de tabela como ordenação, paginação e filtragem. Cada protocolo era exibido com seu código, tipo, cliente, classe, cultura e status atual. As ações disponíveis incluíam edição, exclusão, geração de QR Code e exportação para planilha.

A funcionalidade de geração de QR Code foi implementada com a biblioteca *qrcode* e tinha como objetivo facilitar a identificação dos protocolos em campo. Cada QR Code gerado continha um identificador único que, ao ser escaneado, direcionava o colaborador diretamente para a página do protocolo correspondente na plataforma.

4.2.2 Formulário de Cadastro e Edição de Protocolos

O formulário de cadastro de protocolos representou um dos maiores desafios técnicos do projeto, dado o grande volume de campos e a complexidade das regras de negócio envolvidas. O formulário foi organizado em abas para melhorar a usabilidade: Informações, Avaliações, Tratamentos, Local, Solo, Aplicações, Manejo e Dashboard.

A aba de Informações continha os dados gerais do protocolo, como código, tipo (Rede, Comercial ou Vitrine), cliente, cultura utilizada, datas de plantio e colheita, número de tratamentos e repetições, entre outros campos. Muitos desses campos eram interdependentes: a seleção do tipo de protocolo, por exemplo, determinava quais campos subsequentes seriam obrigatórios ou opcionais.

A aba de Avaliações permitia o registro dos dados coletados em cada momento de avaliação. A estrutura desta aba era dinâmica: o número de colunas da tabela variava de acordo com o número de repetições configurado na aba de Informações, e novas linhas podiam ser adicionadas conforme a necessidade. Além dos dados numéricos, a aba suportava o *upload* de fotografias de campo, permitindo que os técnicos anexassem registros visuais diretamente vinculados a cada momento de avaliação.

O gerenciamento do estado do formulário como um todo foi implementado com as bibliotecas *Formik* e *Immer*. O *Formik* era responsável pelo controle dos valores dos campos e pela orquestração das validações, enquanto o *Immer* facilitava a atualização imutável de estruturas de dados profundamente aninhadas. Essa necessidade era recorrente, dado que um protocolo podia conter dezenas de tratamentos, cada um com múltiplas repetições e avaliações.

Uma das principais dificuldades encontradas neste módulo foi a tradução das regras de negócio da Terra Gerais para a lógica do formulário. As regras variavam conforme o tipo de protocolo, a cultura selecionada e a classe do experimento, exigindo uma comunicação constante com a equipe do cliente para validar o comportamento esperado. O uso da biblioteca *Yup* para validação condicional dos campos mostrou-se essencial para garantir a integridade dos dados antes do envio ao servidor.

4.3 Módulo de Análise e Dashboards

O módulo de análise e dashboards representou o principal diferencial da plataforma em relação ao uso de planilhas. A proposta era transformar os dados brutos registrados nos protocolos em visualizações gráficas que auxiliassem os pesquisadores na tomada de decisões.

4.3.1 Geração Automatizada de Gráficos

A geração de gráficos foi implementada utilizando a biblioteca *ApexCharts*, integrada ao React por meio do pacote *react-apexcharts*. Os gráficos eram gerados automaticamente a partir dos dados registrados nas avaliações de cada protocolo, sem a necessidade de configuração manual por parte do usuário. Foram utilizados principalmente gráficos de barras verticais e horizontais para comparar os resultados entre diferentes tratamentos e momentos de avaliação. Cada gráfico era renderizado individualmente em um *card*, com a opção de download individual.

Uma funcionalidade importante foi a exportação dos gráficos em formato PNG com a identidade visual da Terra Gerais, contendo o logotipo da empresa e a legenda dos dados. Essa funcionalidade foi implementada com a biblioteca *html2canvas*, que realiza a captura de elementos HTML como imagem, combinada com a biblioteca *JSZip* para permitir o download de múltiplos gráficos em um único arquivo compactado.

4.3.2 Dashboard Geral

Além dos gráficos individuais por protocolo, a plataforma contava com um dashboard geral que apresentava uma visão consolidada de todos os protocolos cadastrados, incluindo a distribuição de protocolos por classe, por cultura e por status.

A construção dos dashboards exigiu o uso da biblioteca *mathjs* para a realização de cálculos estatísticos sobre os dados dos protocolos, como médias, desvios e percentuais de distribuição. Os dados eram buscados via API utilizando o *React Query (TanStack Query)*, que gerenciava o cache das requisições e garantia que as visualizações estivessem sempre atualizadas sem a necessidade de recarregamentos manuais da página.

Uma das dificuldades técnicas enfrentadas na construção dos dashboards foi a performance de renderização quando o volume de dados era elevado. A renderização simultânea de múltiplos gráficos com grandes conjuntos de dados causava lentidão perceptível na interface. Para mitigar esse problema, foi adotada uma estratégia de carregamento sob demanda (*lazy loading*), em que os gráficos eram renderizados apenas quando o usuário navegava até a seção correspondente.

5 CONSIDERAÇÕES FINAIS

5.1 Avaliação do Produto Desenvolvido

5.2 Impacto do Estágio na Formação Acadêmica e Profissional

REFERÊNCIAS

Centro de Estudos Avançados em Economia Aplicada; Confederação da Agricultura e Pecuária do Brasil. **PIB do Agronegócio Brasileiro**. 2025. Série histórica e boletins técnicos. Disponível em: <https://www.cepea.esalq.usp.br/br/pib-do-agronegocio-brasileiro.aspx>. Acesso em: 07 mar. 2026.

CHHIPA, J. **ApexCharts: Modern & Interactive Open-source Charts**. 2024. Documentação oficial. Disponível em: <https://apexcharts.com/>. Acesso em: 10 mar. 2026.

Empresa Brasileira de Pesquisa Agropecuária. **Visão 2030: o futuro da agricultura brasileira**. Brasília: Embrapa, 2019. Disponível em: <https://www.embrapa.br/visao/o-futuro-da-agricultura-brasileira>. Acesso em: 07 mar. 2026.

KATO, D. **Zustand: Bear necessities for state management in React**. 2024. Repositório GitHub. Disponível em: <https://github.com/pmndrs/zustand>. Acesso em: 10 mar. 2026.

LINSLEY, T. **TanStack Query: Powerful asynchronous state management**. 2024. Documentação oficial. Disponível em: <https://tanstack.com/query/latest>. Acesso em: 10 mar. 2026.

MASSRUHÁ, S. M. F. S.; LEITE, M. A. de A. *et al.* **Agricultura Digital: pesquisa, desenvolvimento e inovação nas cadeias produtivas**. Brasília: Embrapa, 2020.

Meta Platforms. **React: The library for web and native user interfaces**. 2024. Documentação oficial. Disponível em: <https://react.dev/>. Acesso em: 10 mar. 2026.

Microsoft. **TypeScript: JavaScript With Syntax For Types**. 2024. Documentação oficial. Disponível em: <https://www.typescriptlang.org/>. Acesso em: 10 mar. 2026.

MUI. **Material UI: React components that implement Google's Material Design**. 2024. Documentação oficial. Disponível em: <https://mui.com/>. Acesso em: 10 mar. 2026.

PALMER, J. **Formik: Build forms in React, without the tears**. 2024. Documentação oficial. Disponível em: <https://formik.org/>. Acesso em: 10 mar. 2026.

SCHWABER, K.; SUTHERLAND, J. **The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game**. Scrum.org, 2020. Disponível em: <https://scrumguides.org/scrum-guide.html>. Acesso em: 10 mar. 2026.

Zeeway Soluções Digitais. **Institucional Zeeway**. 2026. Site institucional. Disponível em: <https://zeeway.com.br/>. Acesso em: 07 mar. 2026.